Corrected Java Programming Practical Manual

20 corrected programs — clean, copy-paste friendly source code and sample output.

1. Program Structure and Input using Scanner

Program:

```
import java.util.Scanner;

class GetInputData {
    public static void main(String[] args) {
        int num;
        float fnum;
        String str;

        Scanner in = new Scanner(System.in);

        // Get input String
        System.out.println("Enter a string: ");
        str = in.nextLine();
        System.out.println("Input String is: " + str);

        // Get input Integer
        System.out.println("Enter an integer: ");
        num = in.nextInt();
        System.out.println("Input Integer is: " + num);

        // Get input float number
        System.out.println("Enter a float number: ");
        fnum = in.nextFloat();
        System.out.println("Input Float number is: " + fnum);
    }
}
```

Sample Output:

```
Enter a string:
Chaitanya
Input String is: Chaitanya
Enter an integer:
27
Input Integer is: 27
Enter a float number:
12.56
Input Float number is: 12.56
```

2. Factorial of a Given Number

Program:

```
import java.util.Scanner;

class Factorial {
    public static void main(String[] args) {
        int n, c, fact = 1;

        System.out.println("Enter an integer to calculate its factorial:");
        Scanner in = new Scanner(System.in);
        n = in.nextInt();

        for (c = 1; c <= n; c++) {
            fact = fact * c;
        }

        System.out.println("Factorial of " + n + " is = " + fact);
    }
}
```

Sample Output:

```
Enter an integer to calculate its factorial:
5
Factorial of 5 is = 120
```

3. Prime Numbers in a Given Range

Program:

```
import java.util.Scanner;

class PrimeNumber {
    public static void main(String[] args) {
        int n;
        Scanner s = new Scanner(System.in);

        System.out.println("Enter number:");
        n = s.nextInt();

        for (int i = 2; i <= n; i++) {
            int p = 0;

            for (int j = 2; j <= i / 2; j++) {
                if (i % j == 0) {
                    p = 1;
                    break;
                }
            }

            if (p == 0) {
                System.out.print(i + " ");
            }
        }
    }
}
```

Sample Output:

```
Enter number:
40
2 3 5 7 11 13 17 19 23 29 31 37
```

4. Command-Line Arguments

Program:

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("The command line arguments are:");

        for (int i = 0; i < args.length; i++) {
            System.out.println(args[i]);
        }
    }
}
```

Sample Output:

```
javac Demo.java
java Demo Shraddha
The command line arguments are:
Shraddha
```

5. Sort Array in Ascending and Descending Order

Program:

```
public class SortArray {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 7, 1};
        int temp;

        System.out.println("Original array:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

        // Ascending order
        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] > arr[j]) {
                    temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }

        System.out.println("\nAscending order:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

        // Descending order
        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] < arr[j]) {
                    temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }

        System.out.println("\nDescending order:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Sample Output:

```
Original array:
5 2 8 7 1
Ascending order:
1 2 5 7 8
Descending order:
8 7 5 2 1
```

6. Different Types of Constructors

Program:

```
class Demo5 {
    int a, c;

    Demo5() {
        System.out.println("Default constructor");
    }

    Demo5(int l) {
        a = l;
        System.out.println("Parameterized constructor = " + a);
    }

    public static void main(String[] args) {
        Demo5 d1 = new Demo5();
        Demo5 d2 = new Demo5(15);
    }
}
```

Sample Output:

```
Default constructor
Parameterized constructor = 15
```

7. Method Overloading

Program:

```
class DisplayOverloading {
    public void disp(char c) {
        System.out.println(c);
    }

    public void disp(char c, int num) {
        System.out.println(c + " " + num);
    }
}

class Sample {
    public static void main(String[] args) {
        DisplayOverloading obj = new DisplayOverloading();

        obj.disp('a');
        obj.disp('a', 10);
    }
}
```

Sample Output:

```
a
a 10
```

8. Static Members

Program:

```
public class Stat1 {
    static int a = 1;
    int b = 2;

    static void show1() {
        System.out.println(a);
    }

    void show2() {
        System.out.println(a);
        System.out.println(b);
    }

    public static void main(String[] args) {
        Stat1 s1 = new Stat1();

        s1.show1();
        s1.show2();
    }
}
```

Sample Output:

```
1
1
2
```

9. Package using Two Classes

Program:

```
// File: pack1.java
import addi.Additions;
import addi.Subtraction;

public class Pack1 {
    public static void main(String[] args) {
        Additions b = new Additions();
        b.add(18, 9);

        Subtraction c = new Subtraction();
        c.sub(18, 9);
    }
}

// File: addi/Additions.java
package addi;

public class Additions {
    public void add(int c, int d) {
        int r = c + d;
        System.out.println("Addition is " + r);
    }
}

// File: addi/Subtraction.java
package addi;

public class Subtraction {
    public void sub(int c, int d) {
        int r = c - d;
        System.out.println("Subtraction is " + r);
    }
}

// Compile from the parent folder:
// javac addi/Additions.java addi/Subtraction.java Pack1.java
// Run:
// java Pack1
```

Sample Output:

```
Addition is 27
Subtraction is 9
```

10. Single and Multilevel Inheritance

Program:

```
class X {
    public void methodX() {
        System.out.println("Class X method");
    }
}

class Y extends X {
    public void methodY() {
        System.out.println("Class Y method");
    }
}

class Z extends Y {
    public void methodZ() {
        System.out.println("Class Z method");
    }

    public static void main(String[] args) {
        Z obj = new Z();

        obj.methodX();
        obj.methodY();
        obj.methodZ();
    }
}
```

Sample Output:

```
Class X method
Class Y method
Class Z method
```

11. Method Overriding

Program:

```
class College {
    public void move() {
        System.out.println("College is open");
    }
}

class Univ extends College {
    @Override
    public void move() {
        System.out.println("University is open too");
    }
}

public class Stud {
    public static void main(String[] args) {
        College a = new College();
        College b = new Univ();

        a.move();
        b.move();
    }
}
```

Sample Output:

```
College is open
University is open too
```

12. Interface for Driving Vehicle

Program:

```
interface Driving {
    void accelerate(int speed, String vehicleType);
    void moveForward(int mileage);
    void reverse();
    void turnLeft();
    void turnRight();
}

class Vehicle implements Driving {
    public void accelerate(int speed, String vehicleType) {
        System.out.println(vehicleType + " is moving at " + speed + " kilometers");
    }

    public void moveForward(int mileage) {
        System.out.println("Vehicle is moving forward with mileage " + mileage);
    }

    public void reverse() {
        System.out.println("Vehicle takes reverse direction");
    }

    public void turnLeft() {
        System.out.println("Turning vehicle left");
    }

    public void turnRight() {
        System.out.println("Turning vehicle right");
    }
}

public class DemoV {
    public static void main(String[] args) {
        Vehicle v = new Vehicle();

        v.accelerate(20, "Sports_bike");
        v.turnLeft();
        v.accelerate(10, "Sports_bike");
        v.turnRight();
        v.accelerate(8, "Sports_bike");
        v.reverse();
        v.moveForward(35);
    }
}
```

Sample Output:

```
Sports_bike is moving at 20 kilometers
Turning vehicle left
Sports_bike is moving at 10 kilometers
Turning vehicle right
Sports_bike is moving at 8 kilometers
Vehicle takes reverse direction
Vehicle is moving forward with mileage 35
```

13. Interface and Vehicle Class

Program:

```
import java.util.Scanner;

interface Info {
    void getData();
    void showData();
}

class Vehicle implements Info {
    Scanner s = new Scanner(System.in);

    String vNo;
    String vName;
    String vMake;
    String vColor;

    public void getData() {
        System.out.print("Enter the vehicle number: ");
        vNo = s.nextLine();

        System.out.print("Enter the vehicle name: ");
        vName = s.nextLine();

        System.out.print("Enter the vehicle make: ");
        vMake = s.nextLine();

        System.out.print("Enter the vehicle color: ");
        vColor = s.nextLine();
    }

    public void showData() {
        System.out.println("----- Vehicle Information -----");
        System.out.println("Vehicle No is: " + vNo);
        System.out.println("Vehicle Name is: " + vName);
        System.out.println("Vehicle Make is: " + vMake);
        System.out.println("Vehicle Color is: " + vColor);
    }
}

public class Vehicled {
    public static void main(String[] args) {
        Vehicle v = new Vehicle();

        v.getData();
        v.showData();
    }
}
```

Sample Output:

```
Enter the vehicle number: MH18BP1895
Enter the vehicle name: FZ
Enter the vehicle make: Yamaha
Enter the vehicle color: Magenta
----- Vehicle Information -----
Vehicle No is: MH18BP1895
Vehicle Name is: FZ
Vehicle Make is: Yamaha
Vehicle Color is: Magenta
```

14. Java Streams – Byte Stream and Character Stream

Program:

```
// BYTE STREAM
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class Prog14Byte {
    public static void main(String[] args) {
        try (FileInputStream in = new FileInputStream("input.txt");
             FileOutputStream out = new FileOutputStream("output.txt")) {

            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }

            System.out.println("Data copied to output.txt");
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}

/*
CHARACTER STREAM

import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class Prog14Char {
    public static void main(String[] args) {
        try (FileReader in = new FileReader("input.txt");
             FileWriter out = new FileWriter("output.txt")) {

            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }

            System.out.println("Data copied to output.txt");
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
*/
```

Sample Output:

```
Data copied to output.txt
```

15. Count Lines, Words and Characters in a Text File

Program:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Prog15 {
    public static void main(String[] args) {
        int charCount = 0;
        int wordCount = 0;
        int lineCount = 0;

        try (BufferedReader reader =
                 new BufferedReader(new FileReader("input.txt"))) {

            String currentLine;

            while ((currentLine = reader.readLine()) != null) {
                lineCount++;

                String trimmed = currentLine.trim();

                if (!trimmed.isEmpty()) {
                    String[] words = trimmed.split("\\s+");
                    wordCount += words.length;

                    for (String word : words) {
                        charCount += word.length();
                    }
                }
            }

            System.out.println("Number of characters in file: " + charCount);
            System.out.println("Number of words in file: " + wordCount);
            System.out.println("Number of lines in file: " + lineCount);

        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

Sample Output:

```
Number of characters in file: 68
Number of words in file: 13
Number of lines in file: 2
```

16. Drawing Line, Rectangle, Ellipse and Circle using AWT/Swing

Program:

```
import java.awt.Canvas;
import java.awt.Color;
import java.awt.Graphics;
import javax.swing.JFrame;

public class Prog16 extends Canvas {

    public void paint(Graphics g) {
        g.setColor(Color.RED);
        g.fillRect(50, 50, 100, 100);

        g.setColor(Color.GREEN);
        g.fillOval(50, 200, 100, 50);

        g.setColor(Color.BLUE);
        g.fillOval(50, 300, 100, 100);

        g.setColor(Color.MAGENTA);
        g.drawLine(400, 300, 80, 150);
    }

    public static void main(String[] args) {
        Prog16 m = new Prog16();

        JFrame f = new JFrame("AWT Graphics");
        f.add(m);
        f.setSize(600, 600);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Sample Output:

```
A frame opens and displays a rectangle, ellipse, circle and line.
```

17. Swing/AWT Components and Event Handling

Program:

```
import java.awt.event.*;
import javax.swing.*;

public class Prog17 {
    public static void main(String[] args) {
        JFrame f = new JFrame("Swing Components");

        JTextField t = new JTextField();
        JButton b = new JButton("Click");
        JRadioButton r = new JRadioButton("Class1");
        JCheckBox c = new JCheckBox("Class2");

        String[] s = {"One", "Two", "Three"};
        JComboBox<String> cb = new JComboBox<>(s);
        JScrollBar sb = new JScrollBar();

        f.setLayout(null);

        t.setBounds(50, 50, 150, 25);
        b.setBounds(50, 100, 100, 25);
        r.setBounds(50, 150, 100, 25);
        c.setBounds(50, 200, 100, 25);
        cb.setBounds(50, 250, 100, 25);
        sb.setBounds(50, 300, 100, 70);

        b.addActionListener(e ->
            System.out.println("Button Clicked"));

        t.addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {
                System.out.println("Key Pressed");
            }
        });

        r.addMouseListener(new MouseAdapter() {
            public void mouseEntered(MouseEvent e) {
                System.out.println("Mouse Entered");
            }
        });

        c.addMouseListener(new MouseAdapter() {
            public void mouseExited(MouseEvent e) {
                System.out.println("Mouse Exited");
            }
        });

        cb.addActionListener(e ->
            System.out.println("Combo Clicked"));

        f.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                f.dispose();
            }
        });

        sb.addAdjustmentListener(e ->
            System.out.println("Current Value is " + sb.getValue()));

        f.add(t);
        f.add(b);
        f.add(r);
        f.add(c);
        f.add(cb);
        f.add(sb);

        f.setSize(600, 600);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Sample Output:

```
A window opens with JTextField, JButton, JRadioButton, JCheckBox, JComboBox and JScrollBar.
Events are handled using listener interfaces/classes.
```

18. Thread Class and Runnable Interface

Program:

```
class Count implements Runnable {
    Thread myThread;

    Count() {
        myThread = new Thread(this, "My Runnable Thread");
        System.out.println("My thread created: " + myThread);
        myThread.start();
    }

    public void run() {
        try {
            for (int i = 0; i < 5; i++) {
                System.out.println("Printing the count " + i);
                Thread.sleep(1000);
            }
        } catch (InterruptedException e) {
            System.out.println("My thread interrupted");
        }

        System.out.println("My thread run is over");
    }
}

public class RunnableExample {
    public static void main(String[] args) {
        Count cnt = new Count();

        try {
            while (cnt.myThread.isAlive()) {
                System.out.println(
                    "Main thread will be alive till the child thread is live");
                Thread.sleep(1500);
            }
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted");
        }

        System.out.println("Main thread run is over");
    }
}
```

Sample Output:

```
My thread created: Thread[My Runnable Thread,5,main]
Main thread will be alive till the child thread is live
Printing the count 0
Printing the count 1
Printing the count 2
Printing the count 3
Printing the count 4
My thread run is over
Main thread run is over
```

19. Exception Handling using try-catch-finally

Program:

```
class GFG {
    public static void main(String[] args) {
        int[] arr = new int[4];

        try {
            int i = arr[4];

            System.out.println("Inside try block");
        } catch (ArrayIndexOutOfBoundsException ex) {
            System.out.println("Exception caught in catch block");
        } finally {
            System.out.println("Finally block executed");
        }

        System.out.println("Outside try-catch-finally clause");
    }
}
```

Sample Output:

```
Exception caught in catch block
Finally block executed
Outside try-catch-finally clause
```

20. Multiple Catch Blocks

Program:

```
public class Mult {
    public static void main(String[] args) {
        try {
            int a[] = new int[5];
            a[5] = 30 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception occurs");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBounds Exception occurs");
        } catch (Exception e) {
            System.out.println("Parent Exception occurs");
        }

        System.out.println("Rest of the code");
    }
}
```

Sample Output:

```
Arithmetic Exception occurs
Rest of the code
```